

Prueba Técnica

Analista de Automatización y Procesos

Nombre del candidato

Fecha

Objetivo

Evaluar la capacidad del candidato para comprender un problema operativo, diseñar una solución automatizada, implementarla en Python y entregar una solución funcional, organizada y correctamente documentada.

La evaluación está enfocada principalmente en la capacidad para resolver problemas mediante programación y automatización de procesos.

1. Caso práctico

El área administrativa recibe diariamente un archivo CSV con solicitudes realizadas por diferentes colaboradores. Actualmente, la revisión, clasificación y consolidación de esta información se realiza de forma manual, lo que incrementa el tiempo operativo y el riesgo de errores.

Cada registro del archivo contiene la siguiente información:

Campo	Descripción
id_solicitud	Identificador único de la solicitud.
fecha	Fecha en la que fue registrada la solicitud.
nombre	Nombre del colaborador que realiza la solicitud.
correo	Correo electrónico del solicitante.
tipo_solicitud	Categoría asociada a la solicitud.
valor	Valor económico relacionado con la solicitud, cuando aplique.
estado	Estado actual del trámite.

Cuadro 1: Estructura del archivo de entrada

Los tipos de solicitud permitidos son:

Tipo	Descripción general
Compra	Solicitud relacionada con la adquisición de productos o servicios.
Reembolso	Solicitud de devolución de dinero por un gasto realizado.
Acceso	Solicitud de creación, modificación o habilitación de accesos.
Soporte	Solicitud de atención o asistencia técnica.

Cuadro 2: Tipos de solicitud permitidos

Durante la recepción del archivo pueden presentarse inconsistencias como las siguientes:

El objetivo de la prueba consiste en desarrollar una solución que automatice la lectura, validación, clasificación y generación de resultados a partir de este archivo, reduciendo la intervención manual y permitiendo identificar claramente los registros válidos y aquellos que requieren corrección.

Tipo de inconsistencia	Descripción
Registros duplicados	Solicitudes repetidas con el mismo identificador.
Correos inválidos	Direcciones de correo con formato incorrecto.
Campos vacíos	Información obligatoria que no fue diligenciada.
Valores negativos	Valores económicos menores que cero.
Estados no estandarizados	Estados escritos con diferentes formatos o variaciones.
Filas incompletas	Registros que no contienen todos los campos requeridos.

Cuadro 3: Principales inconsistencias esperadas

2. Actividad solicitada

Desarrolle una solución en Python que procese el archivo recibido y genere automáticamente los resultados necesarios para facilitar la gestión de las solicitudes.

2.1. Lectura y validación

La solución deberá validar la información contenida en el archivo antes de realizar cualquier procesamiento. Como mínimo, se deberán implementar las siguientes reglas:

Los estados resultantes después del proceso de normalización deberán corresponder únicamente a los siguientes valores:

La automatización deberá reconocer variaciones en el uso de mayúsculas, minúsculas y género gramatical. Algunos ejemplos de normalización esperada son:

El candidato podrá definir reglas de validación adicionales cuando considere que mejoran la calidad del proceso. Estas reglas deberán estar claramente documentadas en el archivo `README.md`.

2.2. Procesamiento

La solución debe separar automáticamente la información en dos grupos.

Solicitudes válidas

Aquellas que cumplen todas las validaciones.

Solicitudes con errores

Aquellas que presentan uno o más errores.

Regla de validación	Condición esperada
Identificador obligatorio	El campo <code>id_solicitud</code> debe contener un valor.
Identificador único	No deben existir registros duplicados con el mismo <code>id_solicitud</code> .
Correo electrónico válido	El campo <code>correo</code> debe cumplir con una estructura válida de dirección electrónica.
Tipo de solicitud permitido	El campo <code>tipo_solicitud</code> debe corresponder a una de las categorías definidas en el catálogo permitido.
Valor numérico	El campo <code>valor</code> debe contener únicamente valores numéricos.
Valor no negativo	El campo <code>valor</code> debe ser igual o mayor que cero.
Estado normalizable	El contenido del campo <code>estado</code> debe poder transformarse a uno de los estados estandarizados.

Cuadro 4: Reglas mínimas de validación

Estado normalizado	Descripción
Pendiente	La solicitud fue registrada, pero aún no ha iniciado su gestión.
En proceso	La solicitud se encuentra actualmente en trámite.
Completado	La solicitud fue gestionada y finalizada satisfactoriamente.
Rechazado	La solicitud no fue aprobada o no continuará con su procesamiento.

Cuadro 5: Estados permitidos después de la normalización

Cada registro rechazado debe indicar claramente las causas del rechazo.

Ejemplo:

ID	Error
SOL-101	Correo inválido
SOL-108	Valor negativo
SOL-120	Tipo de solicitud inválido

Cuando un registro presente varios errores deberán registrarse todos.

Valor recibido	Valor normalizado
pendiente	Pendiente
PENDIENTE	Pendiente
en proceso	En proceso
En Proceso	En proceso
completada	Completado
rechazada	Rechazado

Cuadro 6: Ejemplos de normalización de estados

3. Resultados esperados

La automatización deberá generar como mínimo los siguientes archivos.

1. solicitudes__validas.csv

Contendrá únicamente los registros válidos.

2. solicitudes__con__errores.csv

Contendrá únicamente los registros rechazados y una columna adicional llamada:

errores

3. resumen.json

Debe contener como mínimo:

- Fecha de ejecución.
- Total de registros.
- Registros válidos.
- Registros con errores.
- Cantidad de duplicados.
- Cantidad de solicitudes por tipo.
- Cantidad por estado.
- Valor total de solicitudes válidas.

4. Integración con servicio externo

Una vez finalizado el proceso de validación, únicamente las solicitudes clasificadas como válidas deberán enviarse a un servicio externo. Para efectos de esta prueba no es necesario consumir una API real; el comportamiento del servicio deberá ser simulado por el candidato.

La simulación deberá contemplar diferentes escenarios de respuesta, permitiendo evaluar la capacidad de la solución para manejar procesos de integración y recuperación ante fallos.

4.1. Comportamientos esperados

La integración deberá considerar, como mínimo, los siguientes escenarios:

Escenario	Comportamiento esperado
Envío exitoso	La solicitud es procesada correctamente por el servicio externo.
Error temporal	Se presenta una falla transitoria que permite realizar un nuevo intento de envío.
Error definitivo	La solicitud no puede ser procesada y deberá registrarse el motivo del fallo sin volver a intentarlo.

Cuadro 7: Escenarios mínimos de integración

4.2. Requisitos de implementación

La solución desarrollada deberá cumplir, como mínimo, con las siguientes condiciones:

Toda la información relacionada con la ejecución del proceso deberá almacenarse en un archivo de registro denominado:

`log_procesamiento.txt`

El formato del archivo de registro será definido libremente por el candidato, siempre que permita identificar claramente la secuencia de eventos ocurridos durante la ejecución de la automatización.

5. Requisitos técnicos

La solución deberá desarrollarse en Python.

Puede utilizar:

Requisito	Descripción
Registro de eventos	Cada intento de envío deberá quedar registrado indicando el resultado obtenido.
Manejo de excepciones	Las excepciones generadas durante la integración deberán ser controladas para evitar la interrupción del proceso.
Reintentos automáticos	Cuando ocurra un error temporal, la solución deberá realizar al menos un nuevo intento antes de considerar la solicitud como fallida.
Continuidad del procesamiento	El fallo de una solicitud no deberá impedir el procesamiento de las demás solicitudes pendientes.

Cuadro 8: Requisitos mínimos de la integración

- Librería estándar
- Pandas
- Requests
- Pydantic
- Cualquier otra librería que considere necesaria

La solución deberá:

- Ejecutarse desde consola.
- Recibir la ruta del archivo como parámetro.
- Utilizar funciones reutilizables.
- Manejar excepciones.
- No incluir credenciales dentro del código.
- Tener nombres descriptivos.
- Incluir requirements.txt.
- Incluir README.md.

Ejemplo:

```
python main.py --input solicitudes.csv --output resultados/
```

6. Estructura sugerida

Elemento	Descripción
main.py	Punto de entrada de la aplicación.
requirements.txt	Dependencias del proyecto.
README.md	Instrucciones de instalación y ejecución.
solicitudes.csv	Archivo de entrada suministrado al candidato.
src/validaciones.py	Reglas de validación de los datos.
src/procesamiento.py	Lógica principal de procesamiento.
src/integracion.py	Simulación de integración con el servicio externo.
resultados/solicitudes_validas.csv	Registros válidos.
resultados/solicitudes_con_errores.csv	Registros rechazados.
resultados/resumen.json	Resumen consolidado de la ejecución.
resultados/log_procesamiento.txt	Registro de eventos del proceso.

Cuadro 9: Estructura sugerida del proyecto

La organización propuesta es únicamente una recomendación. El candidato podrá utilizar una estructura diferente siempre que la solución sea clara, modular y fácil de mantener.

7. Criterios de evaluación

Criterio	Descripción	Puntaje
Cumplimiento funcional	La solución cumple los requerimientos planteados	35
Calidad del código	Organización, modularidad y legibilidad	20
Manejo de errores	Validaciones, excepciones y continuidad del proceso	15
Pensamiento de automatización	Capacidad para convertir un proceso manual en uno automatizado	15
Documentación	README, instrucciones y estructura del proyecto	10
Pruebas	Casos de prueba y verificación del funcionamiento	5
TOTAL		100

8. Introspeccion

1. ¿Cómo abordó el problema?
2. ¿Qué decisiones de diseño tomó?
3. ¿Qué mejoraría si dispusiera de más tiempo?
4. ¿Cómo automatizaría la ejecución diaria?
5. ¿Cómo desplegaría esta solución en producción?
6. ¿Cómo manejaría credenciales de una API real?
7. ¿Cómo evitaría reprocesar el mismo archivo?
8. ¿Cómo monitorearía la automatización?
9. ¿Qué riesgos identifica en la solución desarrollada?
10. ¿Qué parte considera más sólida de su implementación?

Éxitos.

Esta evaluación busca conocer su forma de pensar, analizar y resolver problemas reales mediante automatización.